

Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl
Informatik, Universität Trier

Zeitplan bis zum Jahresende

08.01.2026	Datenflussanalyse	Übungsblatt „Datenflussanalyse“
13.01.2026	Datenflussanalyse von TRIPLA	
15.01.2026	Projekt IV (Kontrollflussanalyse)	Abgabe Übungsblatt „Datenflussanalyse“
20.01.2026	Code Clone Detection	
22.01.2026	Reverse Engineering, Erkennung von Entwurfsmustern	Abgabe Projekt IV
27.01.2026	Gastvortrag: „Programm Analysis and Transformation from a Practitioner’s Point of View“ (Jonas Eckstein, itestra)	Projekt V (Datenflussanalyse)
29.01.2026	Programmtransformationen	Übungsblatt „Programmtransformationen“
03.02.2026	Partielle Evaluation von TRIPLA	
05.02.2026	Higher-Order TRIPLA	
10.02.2026	Details zum Portfolio + Tipps zu Projekt V	Abgabe + Übung ÜTRAFO
17.03.2026	Abgabe Portfolio inkl. Projekt V	

Reverse Engineering und Erkennung von Entwurfsmustern



Bildquelle: mit KI erzeugt (Bing, DALL-E)



Reverse Engineering ist industrielle Praxis



BYD F8



BENZ CLK



BYD S8 Tail



Peugeot 207CC Tail



Reverse Engineering laut Wikipedia

- **Reverse Engineering** (engl., bedeutet: *umgekehrt entwickeln, rekonstruieren*, Kürzel: *RE*), auch **Nachkonstruktion**, bezeichnet den Vorgang, aus einem **bestehenden, fertigen System** oder einem meistens industriell gefertigten Produkt durch **Untersuchung der Strukturen, Zustände und Verhaltensweisen**, die **Konstruktionselemente zu extrahieren**.
- [...] Speziell **bezogen auf Computer-Software**, wird darunter meistens einer der drei folgenden Vorgänge verstanden:
 - Die Rückgewinnung des **Quellcodes** oder einer vergleichbaren Beschreibung aus Binärcode. Z. B. von einem ausführbaren Programm oder einer Programmbibliothek, etwa mit einem Disassembler (kann Teil eines Debuggers sein) oder einem Decompiler.
 - Die Erschließung der Regeln eines **Kommunikationsprotokolls** aus der Beobachtung der Kommunikation, z. B. mit einem Sniffer.
 - Die nachträgliche Erstellung eines **Modells**, ausgehend von bereits vorliegendem Quellcode, in der objektorientierten Programmierung.

Reverse Engineering von Software

Praxis

- Viele UML Editoren unterstützen die Erstellung von **Modellen aus Quelltext**
 - ArgoUML, EclipseUML, Fujaba, ...
 - Visual Paradigm for UML, Altova UModel, ...
- **Modellgetriebene Softwareentwicklung**
 - **Generierung von Quelltext basierend auf Modellen**
 - Während der gesamten Entwicklungszeit!
 - **Nachteil:** Manuelle Änderungen an generiertem Code gehen verloren gehen bzw. werden nicht berücksichtigt
 - **Lösung:** Roundtrip Engineering
 - Synchronisation zwischen Modell und Quelltext
 - Änderungen an (generiertem) Code wirken sich auf das Modell aus



ERKENNUNG VON ENTWURFSMUSTERN

Bildquelle: mit KI erzeugt (Bing, DALL-E)

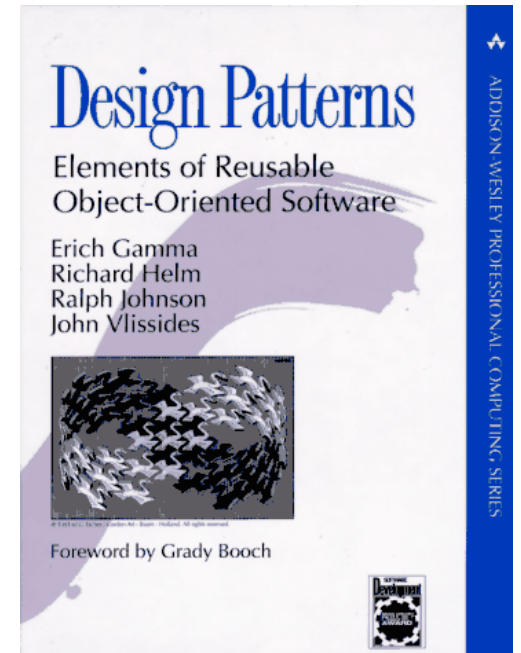
Software Comprehension

- Verstehen von Design und Architektur
 - Automatische Ansätze sollen den Programmierer unterstützen
 - **Problem:** Automatische Erkennung von Design schwierig
 - „Beschränken“ auf die Erkennung von Entwurfsmustern
 - Entwurfsmuster lösen bestimmte Design Probleme
- => Ableiten des beabsichtigten Designs



Entwurfsmuster und Reverse Engineering

- Implementierungen verwenden gut verstandene und definierte Entwurfsmuster
- Musterkatalog von Gamma et al.:
 - *“Wenn man die funktionalen und/oder nicht-funktionalen Eigenschaften X,Y erreichen will, dann sollte man das Muster Z verwenden”*
- Umgekehrt gilt:
 - *Wenn wir ein Muster Z finden, dann waren sehr wahrscheinlich die Eigenschaften X,Y Entwurfsziele.*

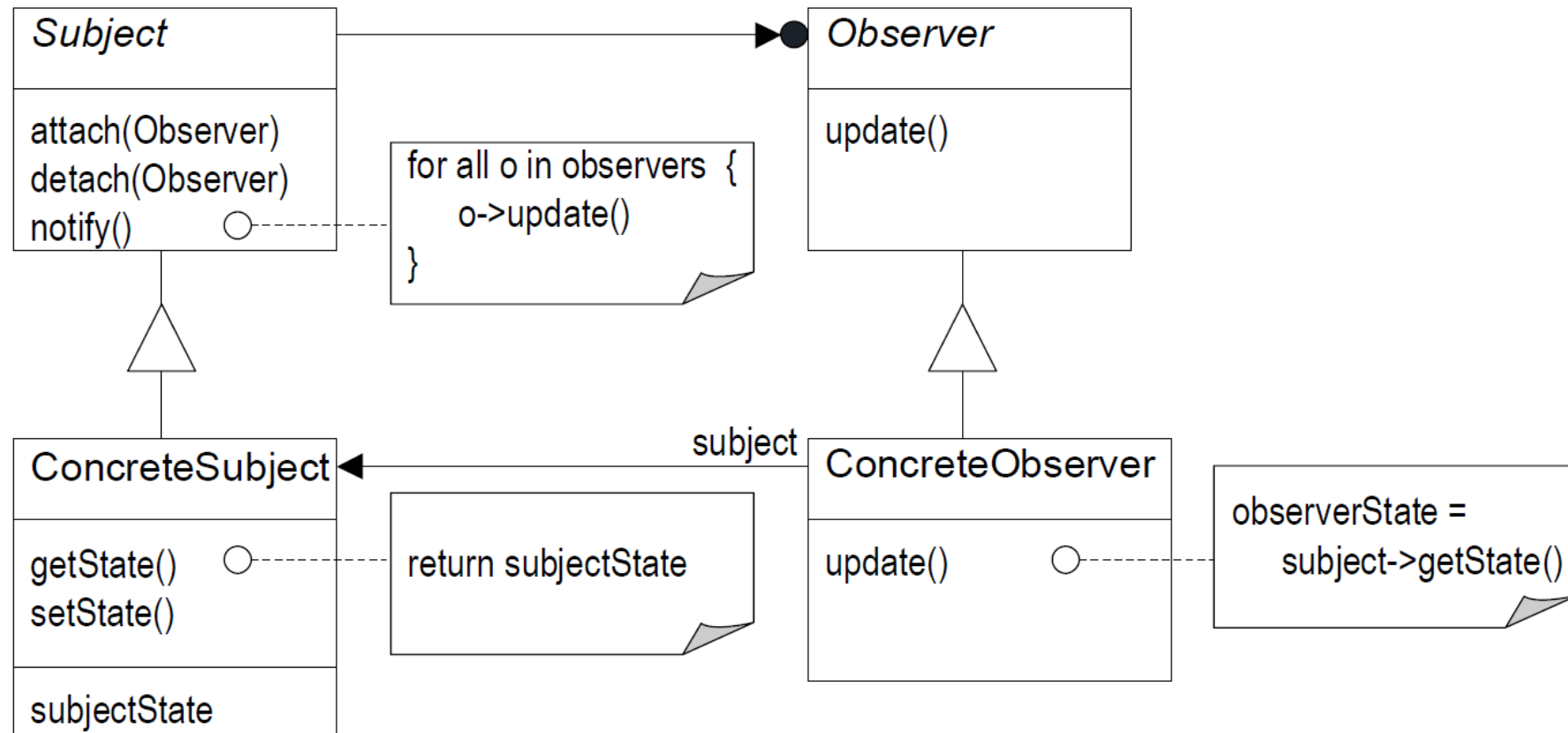


Rekonstruktion/Erkennen von Entwurfsmustern

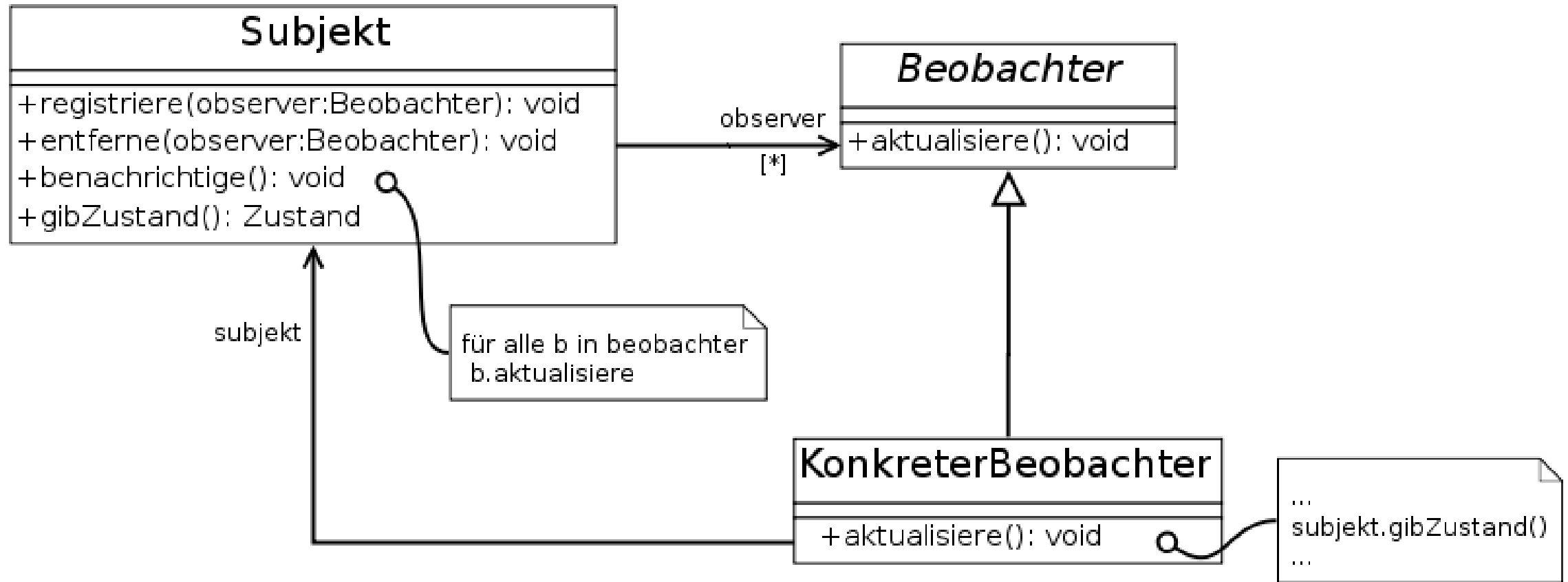
- Definition von Entwurfsmustern:
 - **Struktur:** Klassen, Methoden, Methodenaufrufe, Anweisungen } Statische Eigenschaften
 - **Verhalten:** Folge von Instanziierungen und Methodenaufrufen } Dynamische Eigenschaften
- Entwurfsmustererkennung
 - Suche Menge von Klassen, die der **Struktur** des Musters entsprechen
→ Kandidaten
 - Überprüfe **Verhalten** der Kandidaten



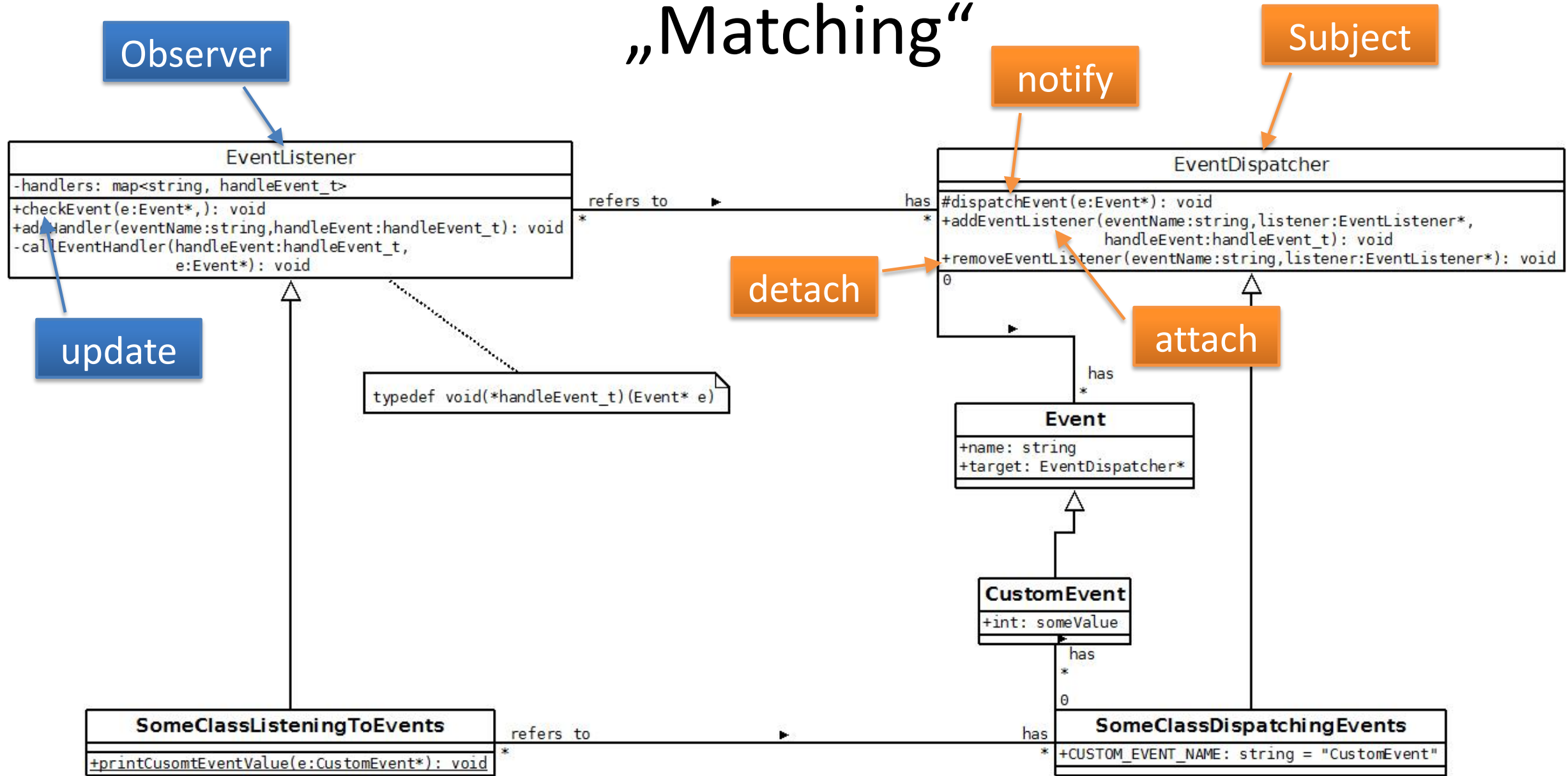
Beispiel: Struktur des Observer-Entwurfsmusters



Beispiel: Struktur des Observer-Entwurfsmusters



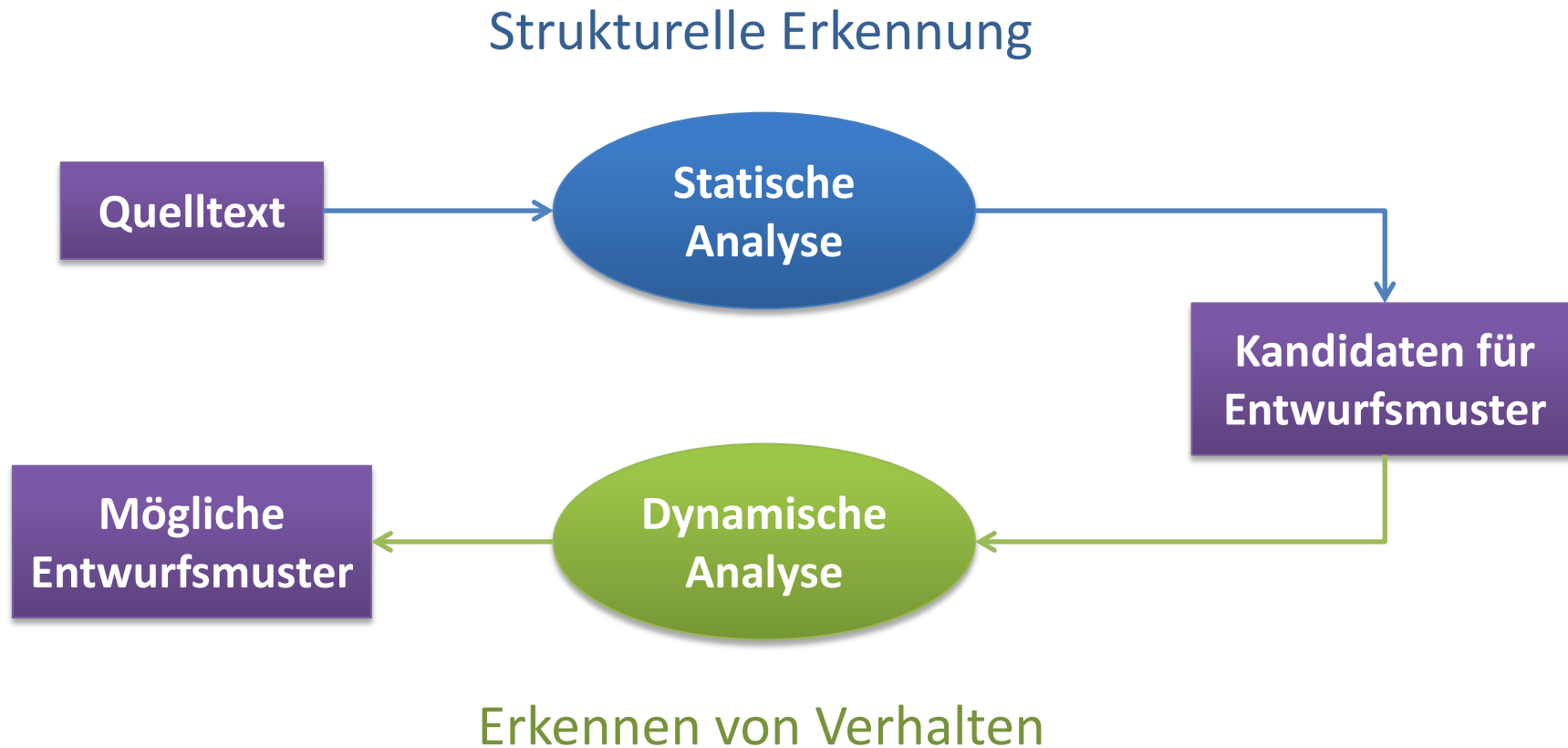
„Matching“



Das Observer-Entwurfsmuster kurz und in Worten

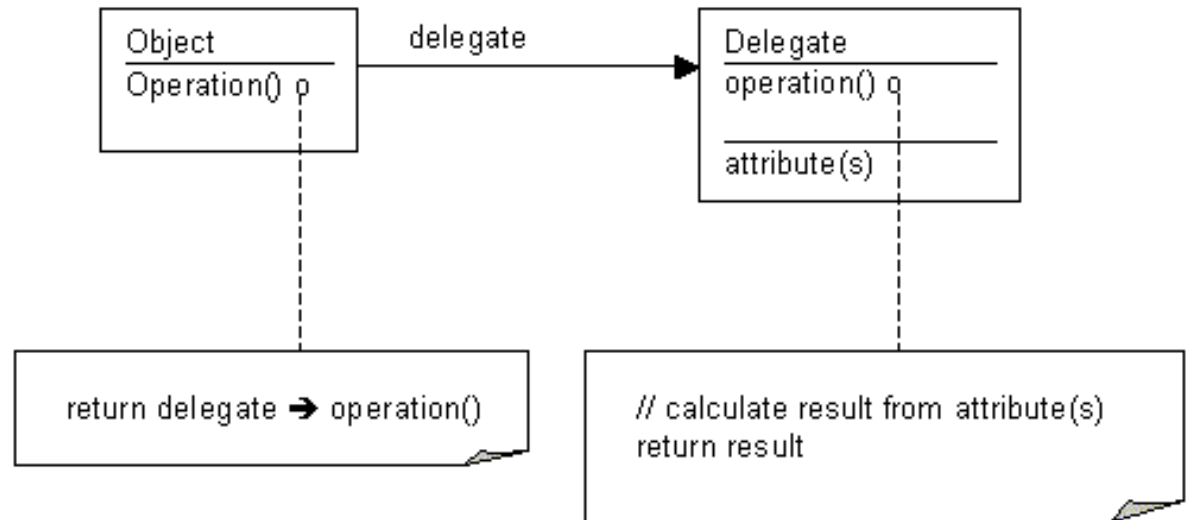
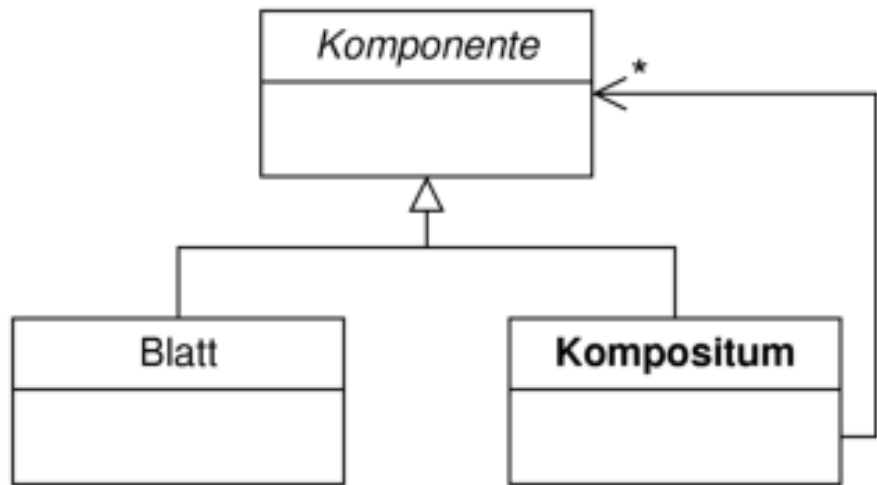
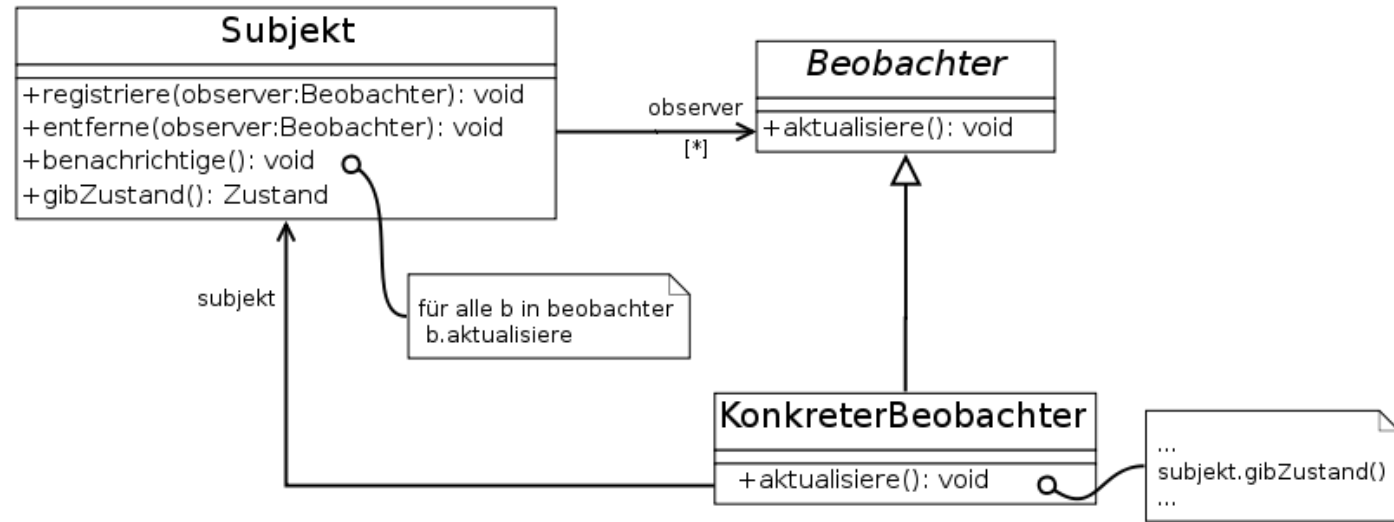
- Ein Objekt
 - erlaubt es anderen Objekten, sich bei ihm **an-** und optional auch **abzumelden**
und
 - **benachrichtigt** ggf. **alle angemeldeten Objekte** durch Aufruf von deren *update*-Methode.

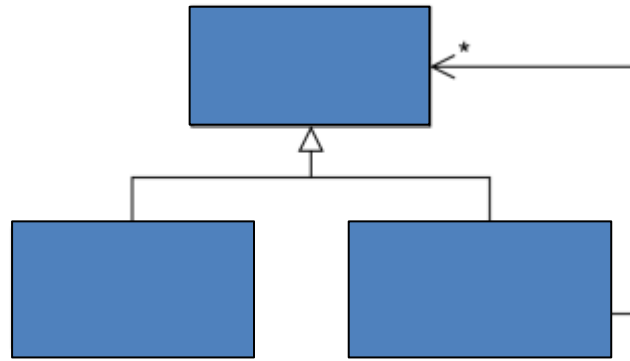
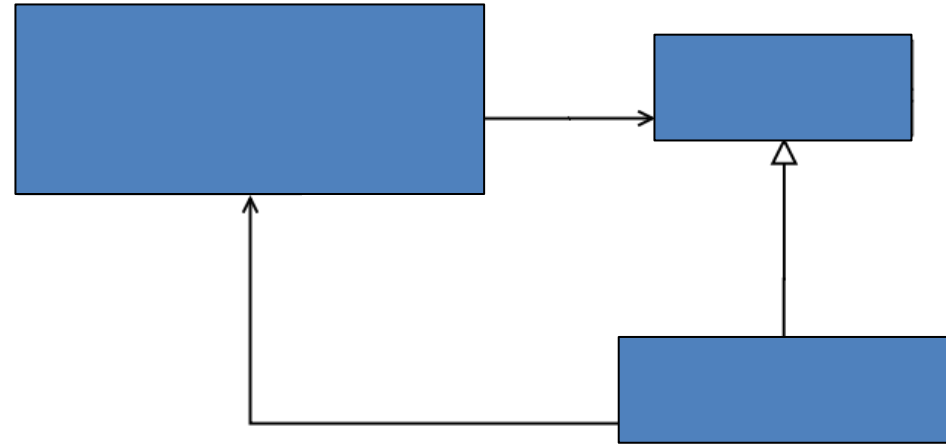
Zweistufiger Ansatz



Strukturelle Erkennung (statisch)

- Erkennung aufgrund von **Namenskonventionen**
 - Benötigt Expertenwissen
 - Gut, um beabsichtigte, aber nicht vollkommen (richtig) umgesetzte Muster zu finden
 - **Problem:** Muster werden häufig unbewusst implementiert oder ohne sich an die Namenskonventionen zu halten.
- Erkennung aufgrund der **Struktur**
 - Gut, um Muster und Anti-Muster zu finden
 - Sehr großer Suchraum!!!





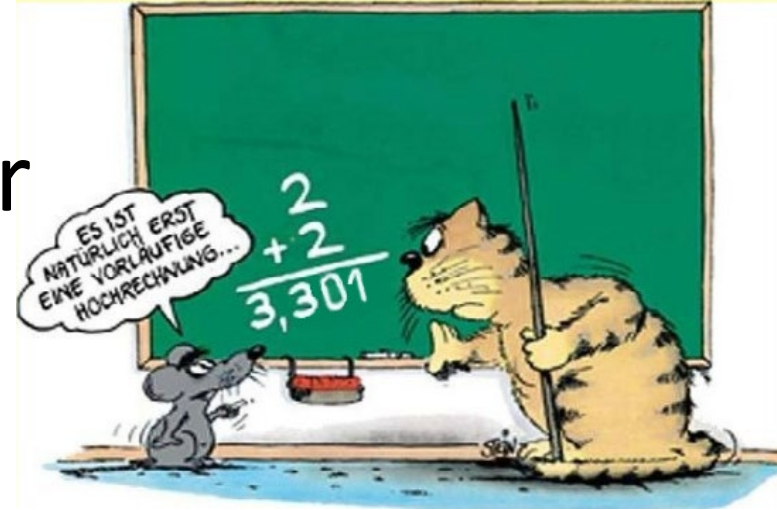
Suchraum

- Jede Klasse der Anwendung ist Kandidat für jede Klasse in jedem Muster.
- Jede Methode der Anwendung ist Kandidat für jede Methode in jedem Muster.
- Die Struktur vieler Muster ist selten sehr charakteristisch
 - vgl. Delegate, Composite und Observer
 - Unterschied ergibt sich durch Verhalten



Große Zahl von Kandidaten; darunter sehr viele falsch-positive.

Beispiel: Suchraum für Observer



- Kandidaten sind Tupel mit 3 Methoden aus einer Klasse und 1 Methode aus einer anderen:
 - *(Subject.attach, Subject.detach, Subject.notify, Observer.update)*
 - *(Subject.attach, _ , Subject.notify, Observer.update)*
- Suchraum für ein System mit n Methoden

» $n = 10$: 210+120
 » $n = 100$: 5.537.925
 » $n = 1000$: 41.583.291.750

Apache Tomcat → $n \approx 15000$
 JDK 7 → $n \approx 100000$

$$\binom{n}{k}$$

Anzahl k -elementiger
Teilmengen einer n -
elementigen Menge

$$\binom{n}{3} + \binom{n}{4}$$

Testfunktionen für Algorithmus zur Kandidatenberechnung (statisch)

- *isNotify(Subject.notify, Observer.update)* ::= true, falls *Subject.notify* ruft *Observer.update* auf und *Observer* ist **kein** Parameter von *notify*
- *isAttach(Subject.attach)* ::= true, falls *Subject.attach* speichert (möglicherweise) das übergebene Argument für spätere Verwendung

⇒ Überprüfe, ob das Argument

attach und *detach* kann man so nicht unterscheiden!

- auf der rechten Seite einer Zuweisung auftaucht, d.h. lokal in einem Objekt gespeichert wird

oder

- als Argument an eine andere Methode übergeben wird (möglicherweise ein Methode zum Abspeichern in einem Container-Objekt).

Beispiel: `list.add(observer)`
`list.remove(observer)`
→ `._.(observer)`

Algorithmus zur Kandidatenberechnung (statisch)

```

C := ∅
for each class c do
  Y := ∅
  for each method m in c do
    for each parameter type p in m where  $p \not\subseteq c \wedge c \not\subseteq p \wedge c \neq p$  do
      for each call from c.n to p.u do
        if isNotify(c.n, p.u)
          Y := Y ∪ {(c.m, c.n, p.u)}

for each (c.a1, c.n1, p1.u1) ∈ Y do
  for each (c.a2, c.n2, p2.u2) ∈ Y where  $c.n_1 = c.n_2 \wedge p_1.u_1 = p_2.u_2$  do
    if isAttach(c.a1)
      if c.a1 = c.a2
        C := C ∪ {(c.a1, -, c.n, p.u)}
      else
        C := C ∪ {(c.a1, c.a2, c.n, p.u)}
  }
```

Hier werden alle Methoden n von c durchlaufen und ggf. auch alle Methoden u von p.

Algorithmus zur Kandidatenberechnung (statisch)

```

 $C := \emptyset$ 
for each class  $c$  do
   $Y := \emptyset$ 
  for each method  $m$  in  $c$  do
    for each parameter type  $p$  in  $m$  where  $p \not\subseteq c \wedge c \not\subseteq p \wedge c \neq p$  do
      for each call from  $c.n$  to  $p.u$  do
        if isNotify( $c.n, p.u$ )
           $Y := Y \cup \{(c.m, c.n, p.u)\}$ 
        update(?)
        notify(?)
  for each  $(c.a_1, c.n_1, p_1.u_1) \in Y$  do
    for each  $(c.a_2, c.n_2, p_2.u_2) \in Y$  where  $c.n_1 = c.n_2 \wedge p_1.u_1 = p_2.u_2$  do
      if isAttach( $c.a_1$ )
        if  $c.a_1 = c.a_2$ 
           $C := C \cup \{(c.a_1, \_, c.n, p.u)\}$ 
        else
           $C := C \cup \{(c.a_1, c.a_2, c.n, p.u)\}$ 
  } Kandidaten
  
```

Diagram annotations:

- Subject (?)** points to c in the first loop.
- attach (?)** points to the *isAttach* call.
- Observer (?)** points to p in the parameter type loop.
- update (?)** points to the *update* call.
- notify (?)** points to the *notify* call.
- Kandidaten** is a green box pointing to the final set C .

Erkennen von Verhalten

- **Ziel:** Erkennung von
 - Folgen von Aufrufen der Methoden von Kandidaten
 - instanziierten Objekten der Kandidatenklassen
- Mögliche Lösungsansätze:
 - nicht notwendig, wenn strukturelle Erkennung kaum/keine falsch-positiven Ergebnisse liefert.
 - verwende Namenskonventionen (Expertenwissen notwendig)
 - Statische Kontrollfluss- und Datenflussanalyse (Überabschätzung, unpräzise aber vollständig)
 - Dynamische Analyse (Testläufe, präzise aber unvollständig)

Dynamische Analyse

- Optimistische Analyse
 - Garantiert, dass Eigenschaften in mindestens einem Lauf korrekt sind.
 - Nicht vollständig: garantiert nicht, dass Eigenschaften für alle Läufe gelten
- Erkennt Übereinstimmungen mit Verhalten eines Musters in einem oder mehreren Läufen
- Wie kann man wirkliche Übereinstimmungen mit einer optimistischen Analyse finden?

Lösungsidee

- Definiere Verletzungen des Musters
- **Wenn eine optimistische Analyse eine solche Verletzung erkennt, dann handelt es sich garantiert um einen falsch-positiven Kandidaten.**
- Beispiel
 - Statt für alle Läufe zu prüfen, ob `object1 == object2` ist, um eine sichere Übereinstimmung zu erkennen,
 - teste, ob es einen Lauf gibt, in dem `object1 != object2` ist, um sicher einen falsch-positiven Kandidaten zu erkennen.

Beispiel: Dynamische Analyse für Verhalten von Observer-Muster

- Jeder **Musterkandidat** c wird gespeichert als:
 $c = (Subject, (attach, detach, notify), Observer.update)$
- Zur Laufzeit
 - Es kann mehr als ein Objekt der Klasse *Subject* geben und jedes dieser Objekte kann mehrere *Observer*-Objekte haben.
- **Laufzeitobjekte** werden gespeichert als:
 $c = \{ (s_1, \{ o_1 \dots o_q \}) \dots (s_n, \{ o_1 \dots o_r \}) \}$

Beispiel: Dynamische Analyse für Verhalten von Observer-Muster

- On *attach* candidate execution:
 - store observer of subject
- On *detach* candidate execution:
 - remove observer of subject
 - Protocol violation if observer not stored
- On *notify* candidate entry:
 - Mark observers of subject as not updated
- On *update* candidate execution:
 - mark observer of subject as updated
- On *notify* candidate exit:
 - Protocol violation if not all observers of subject are marked as updated



Technische Umsetzung: Instrumentierung

Literatur

- Auf ähnliche Weise wurden auch Erkennen für andere Entwurfsmuster implementiert (Mediator, Chain of Responsibility, Visitor, Abstract Factory)
- Siehe: **Automatic design pattern detection**. Dirk Heuzeroth, Thomas Holl, Gustav Högström, Welf Löwe. *In Proc. of the 11th IEEE International Workshop on Program Comprehension, 2003.*

Brainstorming

- Kombination des Ansatzes mit LLM ?

Detection Methodology	Tool/ Author	Year	Analysis Style	R
Database Query Approaches	Rasool <i>et al.</i>	2010	ST, SE	[2]
	D3	2008	ST, BE	[3]
	Marek Vokac	2006	ST	[4]
	SPOOL	1999	ST	[6]
Metrics-Based Approaches	MAISA	2000	ST	[7]
	FUJAPA	2002	ST,BE	[8]
	Antoniol <i>et al.</i>	1998	ST,BE	[9]
	Detten and Becker	2011	ST	[10]
	Uchiyama <i>et al.</i>	2014	ST,BE	[11]
UML Structure, Graph and Matrix Based Approaches	Seemann and Gudenberg	1998	ST,SE	[12]
	DEPAIC++	2002	ST	[13]
	Columbus	2002	ST	[14]
	SSA	2006	ST	[15]
	DP-Miner	2007	ST,BE,SE	[16]
	Dongjin <i>et al.</i>	2015	ST,BE	[17]
Miscellaneous Approaches	Pat	1996	ST	[19]
	PTIDEJ	2001 2004	ST	[20][21]
	CrocoPat	2003	ST	[22]
	SPQR	2003	ST	[23]
	PINOT	2006	ST,BE	[24]
	DeMIMA	2008	ST	[25]
	DPRE	2009	ST	[26]
	MARPLE	2012	ST,BE	[27]
	Sempatrec	2014	ST,SE	[28]
	KT	1996	ST	[29]
	DP++	1998	ST	[30]
	Kim and Boldyreff	2000	ST	[31]
	Heuzeroth <i>et al.</i>	2003	ST,BE	[32]
	Philippow <i>et al.</i>	2005	ST	[33]
	HEDGEHOG	2005	ST,BE,SE	[34]
	Kaczor <i>et al.</i>	2006	ST	[35]
Note: ST: Structural Analysis BE: Behavioral Analysis SE: Semantic Analysis R: Reference				

Quelle: *A Survey on Design Pattern Detection Approaches*,

Mohammed Ghazi Al-Obeidallah,
Miltos Petridis, Stelios Kapetanakis,
International Journal of Software
Engineering (IJSE), 7(3), 2016

https://www.researchgate.net/publication/318504683_A_Survey_on_Design_Pattern_Detection_Approaches

TABLE 2: Summary of detection approaches based on their detection methodology and analysis style.

Vergleich mit LLM

Quelle:

Design pattern recognition: a study of large language models . Pandey, S.K., Chand, S., Horkoff, J. *et al.* *Empirical Software Engineering* **30**, 69 (2025).
<https://doi.org/10.1007/s10664-025-10625-1>

Empirical Software Engineering (2025) 30:69
<https://doi.org/10.1007/s10664-025-10625-1>



Design pattern recognition: a study of large language models

Sushant Kumar Pandey^{1,2} · Sivajeet Chand¹ · Jennifer Horkoff¹ · Mirosław Staron¹ · Mirosław Ochodek² · Darko Durisic²

Accepted: 5 February 2025 / Published online: 18 February 2025
 © The Author(s) 2025

Abstract

Context As Software Engineering (SE) practices evolve due to extensive increases in software size and complexity, the importance of tools to analyze and understand source code grows significantly.

Objective This study aims to evaluate the abilities of Large Language Models (LLMs) in identifying DPs in source code, which can facilitate the development of better Design Pattern Recognition (DPR) tools. We compare the effectiveness of different LLMs in capturing semantic information relevant to the DPR task.

Methods We studied Gang of Four (GoF) DPs from the P-MART repository of curated Java projects. State-of-the-art language models, including Code2Vec, CodeBERT, CodeGPT, **CodeT5**, and RoBERTa, are used to generate embeddings from source code. These embeddings are then used for DPR via a k-nearest neighbors prediction. Precision, recall, and F1-score metrics are computed to evaluate performance.

Results RoBERTa is the top performer, followed by CodeGPT and CodeBERT, which showed mean F1 Scores of 0.91, 0.79, and 0.77, respectively. The results show that LLMs without explicit pre-training can effectively store semantics and syntactic information, which can be used in building better DPR tools.

Conclusion The performance of LLMs in DPR is comparable to existing state-of-the-art methods but with less effort in identifying pattern-specific rules and pre-training. Factors influencing prediction performance in Java files/programs are analyzed. These findings can advance software engineering practices and show the importance and abilities of LLMs for effective DPR in source code.

Keywords Large language model · Design pattern recognition · Software reengineering · Deep learning

1 Introduction

The complexity and scalability of modern software applications is continually evolving. Design patterns (DPs) (Gamma et al. 1995; Kuchana 2004) offer robust and proven solutions.

Communicated by: Steffen Herbold and Hanjong Choi

This article belongs to the Topical Collection: *Predictive Models and Data Analytics in Software Engineering (PROMISE)*.

Extended author information available on the last page of the article

Table 10 Precision comparison of LLMs versus state-of-the-art methods

Design Patterns	Large Language Models + k-NN					State-of-the-art				
	<i>Code2Vec</i>	<i>CodeBERT</i>	<i>CodeGPT</i>	<i>CodeT5</i>	<i>RoBERTa</i>	<i>SparT</i>	<i>DBF</i>	<i>RaM</i>	<i>DPD</i>	<i>GEML</i>
Abstract Factory	0.58	0.81	0.86	0.84	0.93	N/A	0.97	0.50	N/A	N/A
Builder	0.82	0.80	0.77	0.83	1	N/A	0.73	0.52	N/A	N/A
Factory Method	0.71	0.79	0.78	0.72	0.87	N/A	0.96	0.51	0.64	0.81
Prototype	0.64	0.78	0.87	0.82	0.88	0.86	0.43	0.51	1	N/A
Singleton	0.65	0.73	0.68	0.82	0.92	N/A	0.94	0.54	1	0.94
Mean	0.68	0.77	0.79	0.81	0.92	N/A	0.82	0.51	N/A	N/A

Note*: In a gray cell, bold text denotes the highest precision value for a DP, while in a lime-colored cell, bold text indicates the second-highest precision value

Table 11 Recall comparison of LLMs versus State-of-the-art methods

Design Patterns	Large Language Models + k-NN					State-of-the-art				
	<i>Code2Vec</i>	<i>CodeBERT</i>	<i>CodeGPT</i>	<i>CodeT5</i>	<i>RoBERTa</i>	<i>SparT</i>	<i>DBF</i>	<i>RaM</i>	<i>DPD</i>	<i>GEML</i>
Abstract Factory	0.65	0.81	0.86	0.92	0.93	N/A	0.98	0.64	N/A	N/A
Builder	0.82	0.77	0.77	0.67	1	N/A	0.73	0.61	N/A	N/A
Factory Method	0.85	0.78	0.78	0.67	0.77	N/A	0.73	0.62	N/A	0.88
Prototype	0.69	0.78	0.87	0.83	0.84	0.94	0.73	0.64	1	N/A
Singleton	0.61	0.72	0.70	0.58	0.90	N/A	0.82	0.63	1	0.95
Mean	0.72	0.78	0.79	0.73	0.89	N/A	0.78	0.63	N/A	N/A

Note*: In a gray cell, bold text denotes the highest recall value for a DP, while in a lime-colored cell, bold text indicates the second-highest recall value

“The performance of LLMs in DPR is comparable to existing state-of-the-art methods but with less effort in identifying pattern-specific rules and pre-training.”

Trade-off: Explainability vs Flexibility

- Classical methods:
 - More interpretable
 - Deterministic
- LLMs:
 - More flexible
 - Less explainable
 - Harder to trace reasoning

Quelle: Design pattern recognition: a study of large language models . Pandey, S.K., Chand, S., Horkoff, J. *et al. Empirical Software Engineering* 30, 69 (2025). <https://doi.org/10.1007/s10664-025-10625-1>

Classical Design Pattern Detection



LLM-Based Detection



VS

- Rule-Based Logic

- Graph Analysis

- Feature Engineering



High Setup Cost



Fixed & Inflexible

- Zero-Shot & Few-Shot Learning



Contextual Understanding

Adaptive & Flexible



Low Setup Cost



I can handle it!



Less Explainable?